

Competitive Programming

Section 20 Editorial: 0-1 BFS And Shortest Path Variants

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
2	0-1 BFS reminder	4
3	A. Binary Weight Routes	5
3.1	Restatement	5
3.2	Observations	5
3.3	Algorithm	5
3.4	Correctness proof	5
3.5	Complexity	5
3.6	Full C++ solution	6
3.7	Full Python solution	7
4	B. Conveyor Grid	8
4.1	Restatement	8
4.2	Observations	8
4.3	Algorithm	8
4.4	Correctness	9
4.5	Complexity	9
4.6	Full C++ solution	9
4.7	Full Python solution	10
5	C. Warp Maze	11
5.1	Restatement	11
5.2	Observations	11
5.3	Algorithm	11
5.4	Correctness	12
5.5	Complexity	12
5.6	Full C++ solution	12
5.7	Full Python solution	13
6	Common wrong answers	15
7	Stress testing	16
8	What to carry forward	17

1 Editorial overview

Problem	Model
A. Binary Weight Routes	directed graph with 0/1 edges
B. Conveyor Grid	direction-following costs 0, changing direction costs 1
C. Warp Maze	walking costs 0, warping costs 1

2 0-1 BFS reminder

For every relaxation:

```
if edge cost is 0: push_front  
if edge cost is 1: push_back
```

This preserves the same order that Dijkstra's priority queue would produce for distances that only differ by zero or one at each edge.

3 A. Binary Weight Routes

3.1 Restatement

Given a directed graph with edge weights 0 or 1, answer shortest distances from a fixed source to several targets.

3.2 Observations

All queries share one source, so one 0-1 BFS computes all answers.

3.3 Algorithm

```
dist[s] = 0
dq = [s]
while dq not empty:
    u = pop front
    for u -> v with weight w:
        if dist[u] + w < dist[v]:
            update dist[v]
            push front if w=0 else back
```

3.4 Correctness proof

Lemma. Whenever a state is relaxed with a smaller distance, inserting it at the front for cost 0 and back for cost 1 preserves nondecreasing distance order in the deque.

Proof. A cost-0 relaxation has the same distance as the currently popped state, so it belongs with the current smallest layer. A cost-1 relaxation is exactly one layer later, so it belongs after all states of the current layer. $\square$

Theorem. The algorithm computes shortest distances.

Proof. By the lemma, 0-1 BFS processes states in the same distance order as Dijkstra would for weights 0 and 1. Dijkstra's nonnegative-weight correctness therefore applies. $\square$

3.5 Complexity

Time: $O(n + m)$
Memory: $O(n + m)$

3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, s, qn;
    if (!(cin >> n >> m >> s >> qn)) return 0;
    --s;
    vector<vector<pair<int, int>>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u - 1].push_back({v - 1, w});
    }
    vector<int> queries(qn);
    for (int &x : queries) { cin >> x; --x; }

    const int INF = 1e9;
    vector<int> dist(n, INF);
    deque<int> dq;
    dist[s] = 0;
    dq.push_back(s);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto [v, w] : adj[u]) {
            int nd = dist[u] + w;
            if (nd < dist[v]) {
                dist[v] = nd;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    for (int i = 0; i < qn; i++) {
        if (i) cout << ' ';
        cout << (dist[queries[i]] == INF ? -1 : dist[queries[i]]);
    }
    cout << '\n';
    return 0;
}
```

3.7 Full Python solution

```
from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it); m = next(it); s = next(it) - 1; qn = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1; v = next(it) - 1; w = next(it)
        adj[u].append((v, w))
    queries = [next(it) - 1 for _ in range(qn)]
    inf = 10**18
    dist = [inf] * n
    dist[s] = 0
    dq = deque([s])
    while dq:
        u = dq.popleft()
        for v, w in adj[u]:
            nd = dist[u] + w
            if nd < dist[v]:
                dist[v] = nd
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    print(" ".join(str(-1 if dist[x] == inf else dist[x]) for x in queries))

if __name__ == "__main__":
    main()
```

4 B. Conveyor Grid

4.1 Restatement

Moving in the direction written in the current cell is free; moving in another valid direction costs one. Find the minimum cost from top-left to bottom-right.

4.2 Observations

Each cell has up to four outgoing edges. The edge cost is either 0 or 1, so use 0-1 BFS.

Use the cell you are **leaving** to price a move. At (r, c) , the transition to the neighbor in direction dir has cost 0 exactly when $grid[r][c] == dir$; otherwise it costs 1. Reversing that lookup to inspect the destination cell solves a different problem.

State	Transition	Weight	Deque action after an improvement
(r, c)	valid neighbor in its printed direction	0	push front
(r, c)	any other valid neighbor	1	push back

For the sample, $S=(0,0)$ has R, so $(0,1)$ gets distance 0 and goes to the front. The free route initially follows right, right, down; when it must leave the indicated arrow it pays one. The answer is therefore a count of changed directions, not a count of moves.

4.3 Algorithm

```
dist[*][*] = infinity
dist[0][0] = 0
push (0, 0) to the front of a deque

while the deque is nonempty:
    (r, c) = pop front
    for every valid cardinal neighbor (nr, nc) with label dir:
        w = 0 if grid[r][c] == dir else 1
        if dist[r][c] + w < dist[nr][nc]:
            dist[nr][nc] = dist[r][c] + w
            if w == 0: push (nr, nc) to the front
            else:      push (nr, nc) to the back

print dist[n-1][m-1]
```

4.4 Correctness

Every grid route corresponds to a path in this 0/1 graph with the same cost, and every graph path is a valid route. By 0-1 BFS correctness, the distance to the bottom-right cell is the optimum.

4.5 Complexity

There are $n*m$ states and at most four transitions per state. 0-1 BFS runs in $O(n*m)$ time and uses $O(n*m)$ memory.

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<string> grid(n);
    for (auto &row : grid) cin >> row;
    const int INF = 1e9;
    vector<vector<int>> dist(n, vector<int>(m, INF));
    deque<pair<int,int>> dq;
    dist[0][0] = 0;
    dq.push_back({0, 0});
    int dr[4] = {1, -1, 0, 0};
    int dc[4] = {0, 0, -1, 1};
    char ch[4] = {'D', 'U', 'L', 'R'};
    while (!dq.empty()) {
        auto [r, c] = dq.front(); dq.pop_front();
        for (int i = 0; i < 4; i++) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
            int w = (grid[r][c] == ch[i] ? 0 : 1);
            int nd = dist[r][c] + w;
            if (nd < dist[nr][nc]) {
                dist[nr][nc] = nd;
                if (w == 0) dq.push_front({nr, nc});
                else dq.push_back({nr, nc});
            }
        }
    }
    cout << dist[n - 1][m - 1] << '\n';
    return 0;
}
```

4.7 Full Python solution

```
from collections import deque
import sys

DIRS = [(1, 0, "D"), (-1, 0, "U"), (0, -1, "L"), (0, 1, "R")]

def main() -> None:
    input = sys.stdin.readline
    n, m = map(int, input().split())
    grid = [input().strip() for _ in range(n)]
    inf = 10**9
    dist = [[inf] * m for _ in range(n)]
    dist[0][0] = 0
    dq = deque([(0, 0)])
    while dq:
        r, c = dq.popleft()
        for dr, dc, ch in DIRS:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m:
                w = 0 if grid[r][c] == ch else 1
                nd = dist[r][c] + w
                if nd < dist[nr][nc]:
                    dist[nr][nc] = nd
                    if w == 0:
                        dq.appendleft((nr, nc))
                    else:
                        dq.append((nr, nc))
    print(dist[n - 1][m - 1])

if __name__ == "__main__":
    main()
```

5 C. Warp Maze

5.1 Restatement

Walking to a four-neighbor open cell costs 0. Warping to any open cell within Chebyshev distance 2 costs 1. Find the minimum number of warps from S to G .

5.2 Observations

The graph has 0-cost walking edges and 1-cost warp edges. Each cell has at most 29 outgoing transitions: four walks and 25 warp candidates. This is still linear up to a constant.

The state is simply an open cell (r, c) , and $\text{dist}[r][c]$ is the fewest paid warps needed to reach it. Walls never become states. The two action families must remain separate:

Action	Destination	Weight	Deque action after an improvement
walk	one open cardinal neighbor	0	push front
warp	any open cell with $\max(dr , dc) \leq 2$	1	push back

The warp square includes diagonals and also includes the current cell. The latter is harmless: it cannot improve $\text{dist}[r][c]$. It is clearer to iterate all 25 offsets than to introduce a fragile exception. On the sample, free walking explores the start's whole zero-cost component first; one warp then crosses the blocking wall pattern into the component containing G .

5.3 Algorithm

```
find S and G
dist[*][*] = infinity
dist[S] = 0
push S to the front of a deque

while the deque is nonempty:
    (r, c) = pop front
    for each open four-neighbor v of (r, c):
        if dist[r][c] < dist[v]:
            dist[v] = dist[r][c]
            push v to the front
    for every offset dr, dc in [-2, 2]:
        v = (r + dr, c + dc)
        if v is an open in-bounds cell and dist[r][c] + 1 < dist[v]:
```

```

        dist[v] = dist[r][c] + 1
        push v to the back

print -1 if dist[G] is infinity, otherwise dist[G]

```

5.4 Correctness

The modeled graph exactly represents allowed actions and their warp cost. By 0-1 BFS correctness, the computed distance to G is the minimum number of warps.

5.5 Complexity

Each of the $n*m$ cells has at most $4 + 25$ candidate transitions. The constant is fixed, so the running time is $O(n*m)$ and the memory use is $O(n*m)$.

5.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<string> grid(n);
    pair<int,int> start{-1,-1}, goal{-1,-1};
    for (int r = 0; r < n; r++) {
        cin >> grid[r];
        for (int c = 0; c < m; c++) {
            if (grid[r][c] == 'S') start = {r,c};
            if (grid[r][c] == 'G') goal = {r,c};
        }
    }
    const int INF = 1e9;
    vector<vector<int>> dist(n, vector<int>(m, INF));
    deque<pair<int,int>> dq;
    dist[start.first][start.second] = 0;
    dq.push_back(start);
    int wr[4] = {1,-1,0,0}, wc[4] = {0,0,1,-1};
    while (!dq.empty()) {
        auto [r,c] = dq.front(); dq.pop_front();
        int d = dist[r][c];
        for (int i = 0; i < 4; i++) {
            int nr = r + wr[i], nc = c + wc[i];
            if (nr < 0 || nr >= n || nc < 0 || nc >= m || grid[nr][nc] == '#') continue;

```

```

        if (d < dist[nr][nc]) {
            dist[nr][nc] = d;
            dq.push_front({nr,nc});
        }
    }
    for (int dr = -2; dr <= 2; dr++) {
        for (int dc = -2; dc <= 2; dc++) {
            int nr = r + dr, nc = c + dc;
            if (nr < 0 || nr >= n || nc < 0 || nc >= m || grid[nr][nc] == '#') continue;
            if (d + 1 < dist[nr][nc]) {
                dist[nr][nc] = d + 1;
                dq.push_back({nr,nc});
            }
        }
    }
}
int ans = dist[goal.first][goal.second];
cout << (ans == INF ? -1 : ans) << '\n';
return 0;
}

```

5.7 Full Python solution

```

from collections import deque
import sys

def main() -> None:
    input = sys.stdin.readline
    n, m = map(int, input().split())
    grid = [list(input().strip()) for _ in range(n)]
    start = goal = (-1, -1)
    for r in range(n):
        for c in range(m):
            if grid[r][c] == "S":
                start = (r, c)
            elif grid[r][c] == "G":
                goal = (r, c)

    inf = 10**9
    dist = [[inf] * m for _ in range(n)]
    dist[start[0]][start[1]] = 0
    dq = deque([start])
    while dq:
        r, c = dq.popleft()
        d = dist[r][c]
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] != "#" and d < dist[nr][nc]:

```

```

        dist[nr][nc] = d
        dq.appendleft((nr, nc))
    for dr in range(-2, 3):
        for dc in range(-2, 3):
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] != "#" and d + 1 < dist[nr][nc]:
                dist[nr][nc] = d + 1
                dq.append((nr, nc))
    ans = dist[goal[0]][goal[1]]
    print(-1 if ans == inf else ans)

if __name__ == "__main__":
    main()

```

6 Common wrong answers

- Treating all moves as cost 1.
- Using normal BFS on a graph with zero-cost edges.
- Pushing zero-cost edges to the back.
- Forgetting that warps include diagonal and nearby cells.
- Counting number of moves instead of number of paid actions.

7 Stress testing

For small graphs and grids, compare 0-1 BFS against Dijkstra with a heap. The answers must match. Random tests should include disconnected targets, all-zero paths, and cases where a longer route in steps is cheaper in cost.

8 What to carry forward

0-1 BFS is a specialized shortest-path tool. The hard part is often not the deque; it is modeling the state so that every transition has cost exactly 0 or 1.